

```
sed -E 's/^\s*(OP_[A-Z]+),$$/ "\1",/' &&\
echo '};') > opstr.gen.h
```

Once this file is in place, a simple *make* command will make sure the array is generated from the enum if it is missing, or is regenerated if the enum has been updated.

go generate

Go has parallels to both the stringification concept and *make* used in the last example. Since Go has its own build system instead of Makefiles, there is a special annotation called *go:generate* and the *go generate* command to invoke external generators.

Rob Pike, famous programmer and one of the creators of Go, lists its uses in the Go codebase itself in an official blog entry (go.dev/blog/generate):

“... generating Unicode tables in the unicode package, creating efficient methods for encoding and decoding arrays in encoding/gob, producing time zone data in the time package, and so on.”

One tool used in combination with *go generate* is stringer, whose purpose is similar to the enum stringification we saw above.

In the wild

Code generators are everywhere — from UNIX to Go and k8s ecosystems. Here is a quick listing to give you an idea:

- Compiler compilers: lex, yacc, etc
 - Transpilers
 - Build system generators: Automake, CMake, qmake, etc
 - GUI builders: Glade, Qt Designer, etc
 - Configuration generators: update-grub
 - In the Web world
 - CSS pre-processors: SaSS, LESS, Stylus
 - k8s: kompose, kustomize, helm, etc
 - Interface/binding generators: SIP for Python (used for PyQt, wxPython, etc), PyBindGen, SWIG, etc.
 - Flexible: Telosys from Eclipse (DB/model to any kind of code based on templates)
 - protoc with gRPC Go plugin
 - m4 - a general-purpose macro processor from the 70s, part of UNIX
- I'm yet to try some of the above.

Drawbacks

Code generators are not without drawbacks. To begin with, they can generate truly ugly code. They're so mechanical that not even debuggers like *gdb* can help. Outputs of lex and bison are examples for this. Sometimes this is for better performance. But generators in general can be less efficient too. This is because tools have to be general and won't be able to optimise for individual use

cases. (But you can write your own, right?)

Compiler generators are also known for bad error reporting. This could be one reason some major compilers and interpreters use handwritten parsers.

One could argue that the use of generators makes you not know what's happening behind the scenes. But this is not a hard restriction.

The declarative nature of the input to the generators can be an advantage and disadvantage at the same time. Advantage in the sense that it makes sure we have some sort of specification for our APIs, workflows, etc. But the problem is, we might end up ruining those same specs to work around generator limitations.

Some tips

Okay, code generators are helpful. But what can we do on our part to make up for their shortcomings and make the whole process even better? Here are some tips.

- Be aware of the licensing restrictions of the particular tools you use.
- Use tools like *indent* and *gofmt* to reformat the generated code so that it becomes easy to read and debug. This will also get rid of formatting-related compiler warnings.
- Use linters and other static analysis tools to check for code quality and potential bugs.
- Make sure to write and run tests, especially for auto-generated validators. If you're auto-generating tests, check them manually and lock them.
- Although it's an anti-pattern to commit and push generated artefacts, treat auto-generated code like a regular source and include it in the repo. This will be helpful for others trying to build your project on a platform which the generator doesn't support. It'll be helpful to you too when the generator becomes obsolete in future.
- Finally, and most importantly, try to make code generation part of your pipeline, if it makes sense. I'll explain.

Making it part of the pipeline

Is code generation a one-time process or something that is to be done every time you build the software?

For some cases, it doesn't make sense to put the generation in the pipeline. Maybe you are migrating your project from one programming language to another, or choosing a different set of supporting systems so that the configuration files need to be migrated. Here it's clear that you'll be running the converter only once. The original source files are archived, generated files become the new source, and the generator is never run again.

But in some other cases, you are not planning for a migration. Maybe you need something in different formats, or maybe you need boilerplate related to